
Order-Based Rehearsal Learning

Anonymous Authors¹

Abstract

When a machine learning (ML) model forecasts an undesired event, one often seeks a decision to avoid it, known as the *avoiding undesired future (AUF)* problem. Many *rehearsal learning* methods have been proposed for AUF, but they typically rely on an underlying graph structure; learning such a graph from observational data is challenging and can incur substantial estimation error. In this work, we argue for the sufficiency of the *order* structure for AUF problems, and propose the first order-based rehearsal learning method. Although an order is less informative than a graph, it is sufficient to identify the influence of decisions from observational data, highlighting that learning the entire graph is unnecessary. To learn the order, we develop an information-theoretic method that imposes no restrictions on the form of structural functions or the type of noise distributions. For AUF decision-making, we construct an order-based sampler to approximate the influence of decisions and, combined with a surrogate objective for maximizing the post-decision success probability, reduce the AUF task to a differentiable optimization problem. Experiments demonstrate that our order learning method outperforms existing methods, and that our AUF approach not only surpasses methods relying on learned graphs or learned orders, but also matches or even exceeds oracle baselines that are given the true graph.

1. Introduction

Various machine learning (ML) models have demonstrated strong performance on prediction tasks (Voulodimos et al., 2018; Li, 2017; Batmaz et al., 2019). In many real-world scenarios, however, the goal is not merely to predict outcomes but to make better decisions. Specifically, when

an ML model forecasts an undesired event, such as a decline in sales, one may demand a decision to avoid this, which is referred to as the *avoiding undesired future (AUF)* problem (Zhou, 2022). To address this problem, several *rehearsal learning* approaches have been proposed (Qin et al., 2023; Du et al., 2025; Qin et al., 2025; Tao et al., 2025; Anonymity, 2026). These methods typically leverage an underlying graph structure to recommend a decision which maximizes the probability that the outcome lies within a pre-specified desirable region.

However, the aforementioned methods assume access to a given graph structure that characterizes the relations among all variables, which rarely holds in real-world applications. A plausible solution is to learn the graph structure from the observational data, for which plenty of relevant methods have been developed. For example, in the absence of mutual influence relations and latent variables, a directed acyclic graph (DAG) is widely used to characterize the relations among the variables, and numerous causal discovery methods are proposed for learning a DAG from data (Pearl, 2000; Spirtes et al., 2000; Glymour et al., 2019; Zanga et al., 2022). Nevertheless, learning such a graph structure is often challenging and typically requires strong assumptions (Zhang, 2008; Chickering, 2002; Shimizu et al., 2006; Hoyer et al., 2008; Uhler et al., 2013; Zhang et al., 2024; Xie et al., 2024).

Motivated by the inherent difficulty of graph learning, we revisit a fundamental question: is an explicit graph structure truly necessary for decision-making in the AUF problems? In many practical tasks, humans often make effective decisions without full knowledge of the underlying graph structure (Zhou, 2022). This observation suggests that there may exist a structure that is easier to learn than a graph structure, while still being sufficient to support effective decision-making in AUF problems.

In this paper, we highlight the importance of the *order* structure for decision-making in AUF problems, and propose the first order-based rehearsal learning method. An order structure encodes substantially less information than a graph structure; it is therefore natural to expect that learning an order is easier than learning a graph. We show that the influence of decisions can be identified from observational data using only the variable order, establishing the sufficiency of order structure for decision-making in AUF problems.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

To recover the variable order, we develop OLEM (*Order Learning via conditional Entropy Maximization*). OLEM learns the order recursively by repeatedly identifying and removing the “last” variable from the remaining set of variables. Each step is achieved by maximizing a conditional entropy objective, which is simple to implement and readily compatible with a wide range of entropy estimators (Kozachenko & Leonenko, 1987; Sricharan et al., 2013; Berrett et al., 2019). Compared with existing methods for learning the variable order (Bühlmann et al., 2014; Ghoshal & Honorio, 2018; Rolland et al., 2022; Sanchez et al., 2023; Xu et al., 2024), OLEM provides soundness guarantees without requiring restrictive assumptions on a specific form of structural functions or the type of noise distributions. Moreover, OLEM can be implemented in polynomial time with respect to the variable number, offering an efficiency advantage over classical constraint-based discovery methods such as PC and FCI methods (Spirtes et al., 2000; Zhang, 2008).

Building on the learned order, we further develop a decision-making method, OLEM-Rh, which searches for an action that maximizes the probability that the outcome falls within a desired region. Specifically, we employ the conditional flow model (Ardizzone et al., 2019; Winkler et al., 2019) to construct a sampler for the post-decision distribution based on the order, enabling us to approximate the influence of candidate decisions. Combining with the surrogate objective proposed by Qin et al. (2025) for maximizing the desired-region probability, our method reduces AUF decision-making to a differentiable optimization problem which can be solved using standard gradient-based optimization methods. Experiments validate that OLEM outperforms existing order learning methods, and further show that for AUF problems, OLEM-Rh surpasses pipelines that first learn a graph or learn an order using existing methods, while performing comparably to, or even better than, the oracle baselines that take the true graph structure as input.

Our main contributions are summarized as follows:

1. For AUF problems, we highlight the unnecessary of graph structure. We show that the influence of decisions is identifiable from observational data given only a variable order, implying the sufficiency of order structure.
2. We develop a recursive method to learn variable order, which comes with soundness guarantees while requiring milder assumptions than relevant methods.
3. We present a decision-making method based on order by formulating a differentiable optimization problem using a post-decision sampler and a surrogate objective, resulting in the first order-based rehearsal learning method.

2. Problem Formulation

In this section, we introduce the data-generating model and notations for formalizing the AUF problems. We assume there are no latent variables and no feedback, *i.e.*, no cyclic and mutual influence relations.

In this paper, we consider a random vector $\mathbf{V} = (V_1, \dots, V_d)$ generated by the additive model

$$V_i = f_i(\mathbf{Pa}_i) + \varepsilon_i, \quad i \in [d]. \quad (1)$$

$\mathbf{Pa}_i \triangleq \{V_j \mid j \text{ is parent of } i \text{ in } \mathcal{G}\}$, where $\mathcal{G}$ is a directed acyclic graph (DAG) with node set $[d] \triangleq \{1, \dots, d\}$. ε_i -s are independent noise variables. We denote the model above by a triple $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$.

For an AUF decision-making task (Qin et al., 2023), the variables $V_1, \dots, V_d$ are separated into three disjoint parts $\mathbf{X}$, $\mathbf{Y}$, and $\mathbf{Z}$. The random vector $\mathbf{X} = (X_i)_i$ denotes contextual variables, which are observed before decision-making, $\mathbf{Y} = (Y_i)_i$ denotes decision outcome variables, which the decision-makers wish to fall into a desired region $\mathcal{S}$, and are observed after the decision, and $\mathbf{Z} = (Z_i)_i$ denotes intermediate variables, which appear after observing $\mathbf{X}$ and before observing $\mathbf{Y}$. Note that, any variable in $\mathbf{Z}$ or $\mathbf{Y}$ cannot be a parent of a contextual variable X_i , as X_i has been observed before the decisions. Moreover, part of the intermediate variables $\mathbf{Z}$ are alterable, denoted by $\mathbf{Z}_\mathbf{A} = (Z_{a_1}, \dots, Z_{a_{|\mathbf{A}|}})$, where $\mathbf{A} = (a_i)_i$ is the index vector and $|\mathbf{A}|$ denotes its dimensionality. The goal is to recommend an alteration $\mathbf{z}_\mathbf{A} = (z_{a_1}, \dots, z_{a_{|\mathbf{A}|}})$ of $\mathbf{Z}_\mathbf{A}$, so as to increase the probability that the outcome $\mathbf{Y}$ falls into the pre-specified desired region $\mathcal{S}$, given the context $\mathbf{x}$. Here, $\mathcal{S}$ is considered to be a convex polytope, *i.e.* $\mathcal{S} = \{\mathbf{y} \mid \mathbf{M}\mathbf{y} \preceq \mathbf{d}\}$, where the matrix $\mathbf{M}$ and vector $\mathbf{d}$ are given. $\Delta_\mathbf{A}$ denotes the feasible domain of the decision variables $\mathbf{Z}_\mathbf{A}$. Formally, we aim to solve the following optimization problem:

$$\begin{aligned} \max_{\mathbf{Z}_\mathbf{A}} \quad & \Pr\{\mathbf{Y} \in \mathcal{S} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_\mathbf{A} \stackrel{\text{a}}{=} \mathbf{z}_\mathbf{A}\} \\ \text{s.t.} \quad & \mathbf{Z}_\mathbf{A} \in \Delta_\mathbf{A}, \end{aligned} \quad (2)$$

where $\mathbf{Z}_\mathbf{A} \stackrel{\text{a}}{=} \mathbf{z}_\mathbf{A}$ means setting $\mathbf{Z}_\mathbf{A}$ to the alteration $\mathbf{z}_\mathbf{A}$. Note that we do not assume access to the model $\mathcal{M}$. Instead, we only have observational data $\mathbf{D} = \{\mathbf{V}^i\}_i$.

3. Proposed Method

In this section, we argue for the sufficiency of the order structure in the AUF problems and detail our order-learning approach. Further, we propose a decision-making method exploiting an order-based sampler.

3.1. Order Structure

The order structure is introduced in Definition 3.1. Subsequently, we present Prop. 3.1, which implies that given the

order of the true graph $\mathcal{G}$, the influence of decisions in the AUF problems (2) is identifiable from the observational distribution. This result demonstrates that the order structure is sufficient for addressing AUF decision-making tasks.

Definition 3.1 (Order). Given a graph $\mathcal{G}$ with node set $[d]$, a permutation $\pi = (\pi(1), \dots, \pi(d))$ of $1, 2, \dots, d$ is called an order of $\mathcal{G}$, if and only if $\forall i, j \in [d]$, “ j is a descendant of i in $\mathcal{G}$ ” implies “ $\pi^{-1}(i) < \pi^{-1}(j)$ ”. We write $\pi \sim \mathcal{G}$ to mean that π is an order of $\mathcal{G}$.

Proposition 3.1. Given a random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$, suppose $\mathbf{V}$ is separated into $\mathbf{X}, \mathbf{Z}, \mathbf{Y}$ with $\mathbf{Z}_A \subseteq \mathbf{Z}$ according to the AUF setting described in Sec. 2. Given an order $\pi \sim \mathcal{G}$, it holds that

$$\begin{aligned} p(\mathbf{v} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A) \\ = \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) \delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v}) \prod_i p(v_i \mid \text{pred}_i(\pi)), \end{aligned}$$

where the product is taken over the V_i -s that are not contained in $\mathbf{X}$ or $\mathbf{Z}_A$. $\text{Pred}_i(\pi) \triangleq \{V_j \mid \pi^{-1}(j) < \pi^{-1}(i)\}$ denotes the predecessors of V_i according to π , and $\text{pred}_i(\pi)$ denotes an assignment to $\text{Pred}_i(\pi)$. $\delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v})$ is a Dirac function satisfying: (1) $\delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) = 0$ if the sub-vector of $\mathbf{v}$ corresponding to $\mathbf{X}$ is not equal to $\mathbf{x}$, and (2) $\int \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) d\mathbf{v} = 1$. $\delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v})$ is similarly defined.

Remark 3.1. Prop. 3.1 demonstrates that, given the order structure, the post-decision distribution $p(\mathbf{v} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$ is identifiable from the observational distribution. Further, the quantity $\Pr\{\mathbf{Y} \in \mathcal{S} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A\}$, which constitutes the objective of the AUF problems (2), can be derived from observational data. This confirms that the influence of AUF decisions is identifiable using merely the variable order, thereby establishing the sufficiency of the order structure for AUF decision-making.

With the sufficiency of the order established, we first study order learning. We present our method for learning the order in Sec. 3.2, and then introduce the corresponding order-based decision-making method in Sec. 3.3.

3.2. Order Learning via Entropy Maximization

Recall the random vector $\mathbf{V} = (V_1, \dots, V_d)$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$. In this part, we present our method for learning a variable order $\pi \sim \mathcal{G}$. The method proceeds iteratively: at each iteration, it identifies a sink node among the remaining variables and then removes it, where a sink node is a node with no children. Repeating this procedure for d iterations yields an order over all variables, and the overall algorithm can be implemented in time polynomial in d . The key challenge is therefore how to reliably identify a sink node in each iteration.

To this end, we design an information-theoretic criterion for identifying sink nodes, whose soundness is guaranteed

under Assumption 3.1. Notably, this assumption imposes no restrictions on a specific form of structural functions f_i -s or the type of noise distributions, offering strong adaptivity across a wide range of data-generating mechanisms.

Assumption 3.1. Given any random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$, for any $\pi \sim \mathcal{G}$, $m \in [d]$, and for any $i, j \in \pi[1:m]$ such that i is a sink in $\mathcal{G}^{\pi[1:m]}$, while j is not, there holds

$$I(V_j; \text{Des}_j^{\pi[1:m]} \mid \mathbf{W}_j^{\pi[1:m]}) > h(\varepsilon_j) - h(\varepsilon_i),$$

where d denotes the node number of $\mathcal{G}$, $h(\cdot)$ denotes the information entropy, and $I(\cdot; \cdot)$ denotes the mutual information. $\text{Des}_j \triangleq \{V_l \mid l \text{ is descendant of } j \text{ in } \mathcal{G}\}$ denotes the descendants of V_j , and $\mathbf{W}_j \triangleq \{V_1, \dots, V_d\} \setminus (\{V_j\} \cup \text{Des}_j)$ denotes the non-descendants of V_j . The superscript $\pi[1:m]$ means taking only the first m indices in π , i.e., $\mathcal{G}^{\pi[1:m]}$ denotes the induced subgraph of $\mathcal{G}$ over the node set $\pi[1:m] \triangleq \{\pi(i) \mid 1 \leq i \leq m\}$, and $\mathbf{V}^{\pi[1:m]} \triangleq \{V_j \mid j \in \pi[1:m]\}$, $\text{Des}_j^{\pi[1:m]} \triangleq \text{Des}_j \cap \mathbf{V}^{\pi[1:m]}$, $\mathbf{W}_j^{\pi[1:m]} \triangleq \mathbf{W}_j \cap \mathbf{V}^{\pi[1:m]}$.

Remark 3.2. From an information-theoretic view, Assumption 3.1 means that, for each non-sink node j , the variable V_j passes to its descendants a significant amount of information exceeding the noise information gap $h(\varepsilon_j) - h(\varepsilon_i)$, disregarding other information sources $\mathbf{W}_j^{\pi[1:m]}$. This is sensible and mild in practice, since the mutual information term is non-negative, and thus the condition holds whenever $h(\varepsilon_j) < h(\varepsilon_i)$. Additionally, from a communication standpoint, Assumption 3.1 lower bounds the conditional channel capacity between a non-sink variable and its descendants.

Under Assumption 3.1, we present the following proposition, which establishes a criterion for identifying the sink node and serves as the foundation for our proposed order-learning method.

Proposition 3.2. Under Assumption 3.1, for any $\pi \sim \mathcal{G}$ and $m \in [d]$, it holds that

$$i^* = \arg \max_{i \in \pi[1:m]} h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]}) \text{ implies } i^* \text{ is a sink in } \mathcal{G}^{\pi[1:m]},$$

where $\mathbf{V}_{-i}^{\pi[1:m]} \triangleq \{V_j \mid j \in \pi[1:m], j \neq i\}$.

Remark 3.3. Prop. 3.2 implies that the sink node can be identified via a procedure of conditional entropy maximization. Intuitively, for a sink node i , the conditional entropy reduces to the noise entropy, i.e., $h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]}) = h(\varepsilon_i)$. By contrast, for a non-sink node j , conditioning on its non-descendants already yields $h(V_j \mid \mathbf{W}_j^{\pi[1:m]}) = h(\varepsilon_j)$; conditioning on the remaining variables further reduces this quantity. Provided this reduction exceeds the noise entropy gap $h(\varepsilon_j) - h(\varepsilon_i)$, it follows that $h(V_j \mid \mathbf{V}_{-j}^{\pi[1:m]}) < h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]})$, thereby justifying the identification of the sink node by maximizing the conditional entropy.

Algorithm 1 OLEM (Order Learning via conditional Entropy Maximization)

```

1: Input: Joint distribution  $p$  with dimensionality  $d$ 
2: Output: Order  $\pi$ 
3: Initialize  $\Lambda \leftarrow \{1, 2, \dots, d\}$ ,  $\pi = ()$ 
4: for  $t = 1$  to  $d$  do
5:   Compute  $h(\mathbf{V}_{-i}^\Lambda)$  from  $p$  for each  $i \in \Lambda$ 
6:    $i^* \leftarrow \arg \max_i h(V_i | \mathbf{V}_{-i}^\Lambda)$ 
7:    $\pi \leftarrow (i^*, \pi)$  and  $\Lambda \leftarrow \Lambda \setminus \{i^*\}$ 
8: end for
    
```

Leveraging Prop. 3.2, we propose the order-learning strategy outlined in Algorithm 1. This method maintains a set of remaining nodes Λ , and iteratively identifies the sink node i^* by the following:

$$i^* = \arg \max_{i \in \Lambda} h(V_i | \mathbf{V}_{-i}^\Lambda),$$

where $\mathbf{V}_{-i}^\Lambda \triangleq \{V_j | j \in \Lambda, j \neq i\}$. Once identified, i^* is removed from Λ , and the process repeats, until there are no remaining nodes, *i.e.* $\Lambda = \emptyset$.

Algorithm 1 operationalizes the information-theoretic principles of sink-node identification established in Prop. 3.2. Additionally, explicit knowledge of the joint distribution p is not required, as Algorithm 1 relies solely on computing the conditional entropy terms.

In practice, Algorithm 1 is straightforward to implement, since each iteration requires only a conditional entropy maximization step. Furthermore, the method is compatible with various entropy estimators (Kozachenko & Leonenko, 1987; Sricharan et al., 2013; Berrett et al., 2019). For efficient implementation, we observe that $h(V_i | \mathbf{V}_{-i}^\Lambda) = h(\mathbf{V}^\Lambda) - h(\mathbf{V}_{-i}^\Lambda)$. Since the joint entropy $h(\mathbf{V}^\Lambda)$ is constant with respect to i , the optimization problem simplifies to computing $i^* = \arg \min_{i \in \Lambda} h(\mathbf{V}_{-i}^\Lambda)$.

We formally establish the soundness guarantee that Algorithm 1 correctly learns a true order $\pi \sim \mathcal{G}$ under Assumption 3.1, which is stated in Thm. 3.1.

Theorem 3.1. *Let $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ be a model satisfying Assumption 3.1, and let $\mathbf{V}$ denote the random vector generated by $\mathcal{M}$. Given the joint distribution p of $\mathbf{V}$ as input, Algorithm 1 returns an order $\pi \sim \mathcal{G}$.*

Moreover, we position OLEM within a broader landscape by comparing its theoretical scope against relevant algorithms. As summarized in Table 1, existing algorithms often rely on specific constraints regarding the functional form (e.g., linear) or the noise distribution type (e.g., Gaussian). In contrast, OLEM achieves soundness guarantees across all listed categories without assuming a specific form of structural functions or the type of noise distributions.

Table 1. Availability of theoretical guarantees for the listed algorithms under different settings. “L” denotes “Linear”, “G” denotes “Gaussian”, and “n” denotes “non-”.

	CAM	LISTEN	SCORE	DiffAN	CaPS	OLEM
L & G	No	Yes	No	No	Yes	Yes
L & nG	No	No	No	No	No	Yes
nL & G	Yes	No	Yes	Yes	Yes	Yes
nL & nG	No	No	No	Yes	No	Yes

A natural question arises: is this broad adaptivity achieved at the expense of requiring stricter assumptions under simpler settings? For instance, in the linear Gaussian case, does OLEM impose significantly stronger assumptions than specialized methods like LISTEN (Ghoshal & Honorio, 2018) and CaPS (Xu et al., 2024)? To facilitate the comparison, we review the assumptions of LISTEN and CaPS.

Assumption 3.2 (LISTEN (Ghoshal & Honorio, 2018)). Given any random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ under the linear Gaussian setting:

$$f_i(\mathbf{Pa}_i) = \sum_{V_j \in \mathbf{Pa}_i} w_{ji} V_j, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2),$$

for any $\pi \sim \mathcal{G}$, $m \in [d]$, and for any $i, j \in \pi[1 : m]$, if i is a sink in $\mathcal{G}^{\pi[1:m]}$, while j is not, then

$$\sigma_i^{-2} < \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2,$$

where d denotes the node number of $\mathcal{G}$.

Assumption 3.3 (CaPS (Xu et al., 2024)). Given any random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ under the Gaussian setting: $\varepsilon_i \sim \mathcal{N}(0, \sigma_i^2)$, for any $\pi \sim \mathcal{G}$, denoting by d the node number of $\mathcal{G}$, at least one of the following holds.

- (1) For any $i, j \in [d]$, $\pi^{-1}(i) < \pi^{-1}(j)$ implies $\sigma_i < \sigma_j$.
- (2) For any $m \in [d]$, and for any $j \in \pi[1 : m]$, if j is non-sink in $\mathcal{G}^{\pi[1:m]}$, then

$$\sigma_{\min}^{-2} < \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} \mathbb{E}[(\frac{\partial f_l(\mathbf{pa}_l)}{\partial v_j})^2].$$

We present Prop. 3.3 to demonstrate that, when restricted to the linear Gaussian case, the assumption of OLEM reduces to that of LISTEN and is implied by (*i.e.* no stronger than) that of CaPS. This confirms that the adaptivity of OLEM does not come at the cost of stricter assumptions under simpler settings such as linear Gaussian.

Proposition 3.3. *Under the linear Gaussian setting, Assumption 3.1 reduces to Assumption 3.2, and Assumption 3.3 implies Assumption 3.1.*

Algorithm 2 Construction of the sampler for the joint post-decision distribution

```

1: Input: Order  $\pi$ , conditional flow models  $\{T_i\}_{i=1}^d$ ,
   noises  $\{N_i\}_{i=1}^d$ , context  $\mathbf{x}$ , decision  $\mathbf{z}_A$ 
2: Output: Constructed sampler  $T$ 
3: Initialize  $T$  as an empty random vector ( )
4: for  $i = \pi(1)$  to  $\pi(d)$  do
5:   if  $V_i$  is contextual, i.e.  $\exists j, V_i = X_j$  then
6:      $T := (T, x_j)$ 
7:   else if  $V_i$  is alterable, i.e.  $\exists j, V_i = Z_{a_j}$  then
8:      $T := (T, z_{a_j})$ 
9:   else
10:     $T := (T, T_i(N_i, T))$ 
11:   end if
12: end for
    
```

3.3. Order-Based Decision-Making

In this part, we propose an order-based decision-making method for addressing the AUF problem (2). At first, we leverage conditional flow models (Ardizzone et al., 2019; Winkler et al., 2019) to construct a sampler for the post-decision distribution $p_{\mathbf{Y} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A=\mathbf{z}_A}(\cdot | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$ based on the learned order. The sampler further enables approximating the objective function in (2).

We here detail the construction of the sampler. As presented in Prop. 3.1, given the order, the joint post-decision distribution $p_{\mathbf{V} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}$ is identifiable from the observational distribution. Further, direct estimation of the marginal distribution $p_{\mathbf{Y} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}$ typically involves an intractable integral form. To avoid this, we conduct a two-step strategy: (1) we construct a sampler for the joint distribution $p_{\mathbf{V} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}(\cdot | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$ and (2) we extract the components corresponding to $\mathbf{Y}$ to obtain a sampler for $p_{\mathbf{Y} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}(\cdot | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$.

The core challenge lies in step (1), for which we propose the following implementation. Given an order π , we associate each variable V_i with a conditional flow model $T_i(N_i, \text{pred}_i(\pi))$ (Winkler et al., 2019), which transforms a standard Gaussian noise $N_i \sim \mathcal{N}(0, 1)$ and the values of the predecessors $\text{pred}_i(\pi)$ into a variable following the conditional distribution of V_i , *i.e.* $T_i(N_i, \text{pred}_i(\pi)) \sim p_{V_i | \text{pred}_i(\pi)}(\cdot | \text{pred}_i(\pi))$. Subsequently, we employ Algorithm 2 to construct a joint sampler $T(\mathbf{N}, \mathbf{x}, \mathbf{z}_A)$ by composing the individual T_i -s according to π , where $\mathbf{N} \triangleq (N_i)_i$ denotes the joint noise vector. Based on Prop. 3.1 and the aforementioned property of T_i -s, this construction ensures that $T(\mathbf{N}, \mathbf{x}, \mathbf{z}_A) \sim p_{\mathbf{V} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}(\cdot | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$ when $\mathbf{N} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. Notably, the structure of the constructed sampler remains valid for varying contexts $\mathbf{x}$ and decisions $\mathbf{z}_A$ without the need for reconstruction.

Completing the procedure with step (2), we extract the components of the joint sampler $T(\mathbf{N}, \mathbf{x}, \mathbf{z}_A)$ corresponding to the target variables $\mathbf{Y}$. This yields the marginal sampler $T_{\mathbf{Y}}(\mathbf{N}, \mathbf{x}, \mathbf{z}_A) \sim p_{\mathbf{Y} | \mathbf{X}=\mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A}(\cdot | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A)$, where $\mathbf{N} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

To learn the conditional flow models T_i -s, various methods are available. In this paper we employ the conditional invertible neural network (cINN) (Ardizzone et al., 2019) to fit each T_i by minimizing the empirical KL divergence between the output and the target distribution.

Leveraging the constructed sampler, we can approximate the objective of problem (2) and subsequently solve the problem. Specifically, the sampler allows us to express the objective as $\Pr\{T_{\mathbf{Y}}(\mathbf{N}, \mathbf{x}, \mathbf{z}_A) \in \mathcal{S}\}$. Consequently, for any candidate decision $\mathbf{z}_A$, we can estimate the objective value by sampling, *i.e.*

$$\begin{aligned}
 & \Pr\{\mathbf{Y} \in \mathcal{S} | \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A\} \\
 &= \Pr\{T_{\mathbf{Y}}(\mathbf{N}, \mathbf{x}, \mathbf{z}_A) \in \mathcal{S}\} \\
 &\approx \frac{1}{n} \sum_{i=1}^n \mathbb{I}[T_{\mathbf{Y}}(\mathbf{N}^i, \mathbf{x}, \mathbf{z}_A) \in \mathcal{S}],
 \end{aligned}$$

where $\{\mathbf{N}^i\}_{i=1}^n$ are i.i.d. samples of $\mathbf{N}$, and $\mathbb{I}[\cdot]$ denotes the indicator function.

This estimation allows for solving problem (2) using zeroth-order optimization algorithms, such as Bayesian Optimization (Gardner et al., 2014), yet these methods are often computationally inefficient. To address this, we adopt the surrogate objective introduced by Qin et al. (2025). In this manner, we convert the problem into a differentiable optimization task, which can be solved via efficient gradient-based methods. The converted problem is given by:

$$\begin{aligned}
 & \min_{\mathbf{z}_A} \quad \frac{1}{n} \sum_{i=1}^n \|T_{\mathbf{Y}}(\mathbf{N}^i, \mathbf{x}, \mathbf{z}_A) - \mathbf{c}\|_1 \\
 & \text{s.t.} \quad \mathbf{z}_A \in \Delta_A,
 \end{aligned} \tag{3}$$

where $\mathbf{c}$ denotes the Chebyshev center of $\mathcal{S}$, and is computable by linear programming (Boyd & Vandenberghe, 2004). Since $T_{\mathbf{Y}}$ is instantiated using differentiable neural networks, the objective in (3) is fully differentiable with respect to $\mathbf{z}_A$. This allows us to solve the problem using gradient-based optimizers; in our implementation, we employ Adam (Kingma & Ba, 2015).

By integrating the proposed decision-making procedure with the order learned via Algorithm 1, we establish the method OLEM-Rh, detailed in Algorithm 3. This method leverages observational data to learn the variable order and subsequently optimize decisions for the AUF problem. A key strength of OLEM-Rh is its modular architecture, which allows several components to be substituted with alternative techniques, providing strong flexibility for the method.

Algorithm 3 OLEM-Rh

-
- 1: **Input:** Desired region $\mathcal{S}$ parameterized by $\mathbf{M}, \mathbf{d}$, context $\mathbf{x}$, alterable variable indices $\mathbf{A}$, domain $\Delta_{\mathbf{A}}$, observational data $\mathbf{D} = \{\mathbf{V}^i\}_i$
 - 2: **Output:** Decision $\mathbf{z}_{\mathbf{A}}^*$
 - 3: Learn an order π via [Algorithm 1](#), where conditional entropy terms are estimated with $\mathbf{D}$
 - 4: **for** $i = 1$ **to** d **do**
 - 5: Fit the conditional flow model T_i using a cINN trained with π and $\mathbf{D}$
 - 6: **end for**
 - 7: Construct T via [Algorithm 2](#), and further obtain $T_{\mathbf{Y}}$
 - 8: Compute $\mathbf{c}$ from $\mathbf{M}, \mathbf{d}$, and sample $\{\mathbf{N}^i\}_{i=1}^n \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(\mathbf{0}, \mathbf{I})$
 - 9: Obtain $\mathbf{z}_{\mathbf{A}}^*$ by solving (3) using the Adam optimizer with $T_{\mathbf{Y}}, \mathbf{c}, \{\mathbf{N}^i\}_{i=1}^n, \mathbf{x}, \mathbf{A}, \Delta_{\mathbf{A}}$
-

For instance, the specific gradient-based optimizer in line 9 can be varied. More fundamentally, the optimization strategy (lines 8-9) can be replaced by directly solving the primal AUF problem (2) using zeroth-order (black-box) methods, such as Bayesian Optimization ([Gardner et al., 2014](#)). Similarly, the conditional flow models T_i -s (line 5) can be instantiated using other generative models, provided that they satisfy the differentiability requirements of the chosen optimization method.

Discussion. The order structure leveraged in this work aligns with the concept of causal order. To the best of our knowledge, the concept of causal ordering traces back to the 1950s, most notably in the seminal work of [Simon \(1953\)](#). Simon formalized the asymmetric relations in econometrics, distinguishing directional influence from symmetric algebraic equations. This foundational framework later transcended social sciences, inspiring research in qualitative physics ([Iwasaki & Simon, 1986](#)) and artificial intelligence ([Pearl, 2000](#)). In recent years, causal ordering has seen a resurgence, primarily serving as an intermediate representation for learning causal graphs ([Teyssier & Koller, 2005](#); [Bühlmann et al., 2014](#); [Ghoshal & Honorio, 2018](#); [Rolland et al., 2022](#); [Sanchez et al., 2023](#); [Xu et al., 2024](#)). In this work, however, we argue that the order structure itself holds intrinsic value for decision-making, specifically for AUF tasks. We demonstrate that the topological properties of causal order are directly beneficial for these problems and propose the first order-based method tailored for AUF.

4. Experiments

In this section, we conduct experiments to validate the effectiveness of the proposed methods. In [Sec. 4.1](#), we evaluate OLEM for learning the order structure. In [Sec. 4.2](#), we then assess OLEM-Rh on AUF decision-making.

4.1. Order Learning

We compare OLEM with five representative baselines, including LISTEN ([Ghoshal & Honorio, 2018](#)) (primarily designed for linear settings), DiffAN ([Sanchez et al., 2023](#)) (for nonlinear settings), CAM ([Bühlmann et al., 2014](#)) and SCORE ([Rolland et al., 2022](#)) (tailored for nonlinear Gaussian cases), and CaPS ([Xu et al., 2024](#)) (for linear or nonlinear Gaussian cases).

We assess the quality of the learned order using the order divergence (DIV) ([Rolland et al., 2022](#)). Since most baselines return a graph besides an order, we additionally report the Structural Hamming Distance (SHD) and the Structural Intervention Distance (SID) ([Peters & Bühlmann, 2015](#)) of the returned graphs. To compute graph-based metrics for OLEM, we first convert the learned order into a DAG by applying the CAM pruning procedure ([Bühlmann et al., 2014](#)) with cutoff 0.001, and then evaluate SHD and SID on the resulting graph.

We conduct experiments on both synthetic and real-world data. For synthetic evaluation, we generate datasets by varying the dimensionality d , edge density p , linearity r , and noise type. Specifically, we set $d \in \{5, 10, 20\}$ and generate true graphs using the Erdős-Rényi model ([Erdős & Rényi, 1960](#)) with edge density $p \in \{0.3, 0.5, 0.8\}$. The linearity of each structural function is controlled by $r \in \{0, 0.25, 0.5, 0.75, 1\}$: each f_i is linear with probability r and nonlinear otherwise. Linear coefficients are sampled uniformly from $[-1, -0.25] \cup [0.25, 1]$ to keep them bounded away from zero, while nonlinear functions are generated by sampling from Gaussian processes with a unit bandwidth RBF kernel. Noise terms are drawn from beta, exponential, or Gaussian distributions, with distribution parameters sampled uniformly from $[0.75, 1.25]$. The observational sample size is 2000. To reduce instance-specific variance, we adopt a hierarchical evaluation protocol: For each synthetic setting, we report the mean and standard deviation over 5 independent epochs; within each epoch, we repeatedly generate tasks with different ground-truth models and average the metrics.

For real-world evaluation, we use the Sachs dataset ([Sachs et al., 2005](#)), a protein signaling network with 11 nodes and 17 edges, containing 853 samples. We randomly shuffle the dataset in each run and report the mean and standard deviation of each metric over 10 independent runs.

[Figure 1](#) reports results on synthetic datasets with beta or Gaussian noise ($d = 10, p = 0.3$). In the Gaussian setting, CaPS and SCORE achieve the strongest performance, which aligns with their Gaussian noise assumptions.

Under the non-Gaussian noise setting, OLEM consistently performs better than these Gaussian-oriented methods and shows a clear margin over the remaining baselines. This

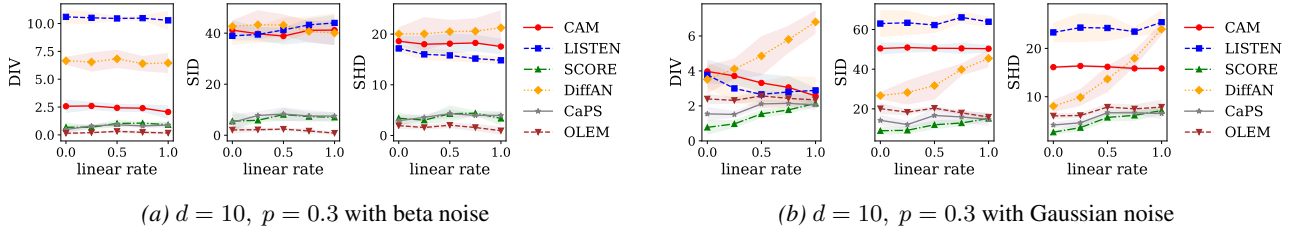

 Figure 1. Results on the synthetic datasets with beta or Gaussian noise ($d = 10, p = 0.3$)

Table 2. Results on the real dataset Sachs. For each dataset and each metric, the best results are shown in bold.

Dataset	Sachs		
Method	DIV↓	SHD↓	SID↓
CAM	8.0±0.0	17.0±0.0	45.0±0.0
LISTEN	8.0±0.0	30.8±6.7	54.1±4.1
SCORE	8.0±0.0	17.0±0.0	45.0±0.0
DiffAN	7.2±1.9	17.8±2.9	49.7±4.4
CaPS	9.0±0.0	16.0±0.0	46.0±0.0
OLEM	8.0±0.0	16.0±0.0	45.0±0.0

demonstrates that OLEM adapts well to diverse data-generating distributions rather than benefiting from a specific noise model. Results for additional dimensionalities, edge densities, and noise types are provided in the appendix.

Table 2 summarizes the results on the Sachs dataset. OLEM achieves the best SHD and SID, while ranking second to DiffAN in terms of DIV. Notably, although DiffAN attains the best mean DIV, it also exhibits the worst mean SID and large variance across data shuffles. This suggests that DiffAN may capture certain ordering-related statistics, whereas OLEM yields a more robust and reliable recovery of the underlying structure.

4.2. AUF Decision-Making

We now evaluate OLEM-Rh on the AUF decision-making problem and focus on three questions:

- (1) Whether AUF decision-making benefits more from learning an explicit graph and using Grad-Rh (Qin et al., 2025) or from learning an order and using OLEM-Rh, *i.e.*, whether an explicit graph is necessary for AUF decision-making?
- (2) Whether OLEM is a crucial component for the order-learning stage or OLEM-Rh remains competitive when coupled with alternative order-learning methods?
- (3) How close OLEM-Rh can get to oracle decision-makers such as Grad-Rh and QWZ23 (Qin et al., 2023) that assume access to the true graph?

Decision quality is measured by the estimated success probability of avoiding the undesired future. Specifically, for each recommended decision, we simulate the decision on the ground-truth model using Monte Carlo evaluation with 1000 independent trials, and use the resulting empirical success rate as the metric.

We conduct experiments on both synthetic and real-world data. For synthetic evaluation, we consider both linear and nonlinear data-generating processes with beta and exponential noise. For each synthetic setting, we construct 20 tasks with distinct true models, alterable-variable sets, domains of decision variables, and desired regions. For each task, we run 50 independent decision rounds and compute the average success probability; we then report the mean and standard deviation across the 20 tasks.

The true graphs are generated using the Erdős-Rényi model (Erdős & Rényi, 1960) with edge probability $p = 0.3$. In linear settings, edge weights are sampled uniformly from $[-1, 1]$. In nonlinear settings, structural functions are instantiated using three-layer MLPs with sigmoid activations, where weights are sampled uniformly from $[-2, -0.5] \cup [0.5, 2]$. Noise generation follows Sec. 4.1. The set of alterable variables is the union of randomly selected ancestors for each Y_i (where each ancestor is selected with probability 0.5). For each alterable variable Z_{a_i} , the decision domain is $[\mu - 2\sigma, \mu + 2\sigma]$, and for each outcome Y_i , the desired region is $[\mu - \sigma, \mu + \sigma]$, where μ and σ denote the empirical mean and standard deviation of the corresponding variable. We set the dimensionality to 15 in linear settings and 12 in nonlinear settings.

For real-world evaluation, we use the Bermuda dataset (Courtney et al., 2017; Andersson & Bates, 2018), which contains environmental measurements from the Bermuda region. Following (Qin et al., 2023), we employ their fitted linear Gaussian simulators to generate 10 pseudo-real datasets. We then follow the same evaluation protocol as in the synthetic experiments: For each dataset, we average the success probability over 50 decision rounds, and report the mean and standard deviation across the 10 datasets.

To assess whether an explicit graph is necessary for AUF decisions, we compare OLEM-Rh to pipelines that first

Table 3. Success probability on four synthetic settings and Bermuda. Methods in the first part learn the order or graph structure from observational data. Methods in the second part take the true graph structure as input. Within each part, results with the highest mean success probability are shown in bold. The sample size is 1000.

	SynLrBeta	SynLrExp	SynNlrBeta	SynNlrExp	Bermuda
OLEM-Rh	.772±.130	.785±.089	.820±.173	.844±.085	.668±.006
Grad-Rh w/OLEM+prun	.738±.196	.761±.193	.816±.155	.840±.082	.668±.006
Grad-Rh w/CaPS	.712±.210	.705±.220	.795±.213	.839±.089	.575±.107
Grad-Rh w/NOTEARS	.453±.264	.396±.247	.795±.165	.831±.078	.322±.021
Grad-Rh w/GES	.654±.223	.590±.270	.786±.222	.829±.081	.279±.144
OLEM-Rh w/CaPS	.752±.148	.750±.121	.802±.180	.808±.126	.668±.005
Grad-Rh (true graph)	.793±.113	.828±.075	.843±.176	.851±.071	.669±.007
QWZ23 (true graph)	.754±.096	.768±.205	.640±.284	.804±.201	.669±.008

learn a graph from observational data and then apply Grad-Rh, which was originally formulated under access to the true graph (Qin et al., 2025). Concretely, we pair Grad-Rh with several structure learning algorithms, including OLEM+prun (where we apply CAM pruning (Bühlmann et al., 2014) to the order learned by OLEM to obtain a graph), CaPS (Xu et al., 2024), NOTEARS (Zheng et al., 2018), and GES (Chickering, 2002). As shown in Table 3, across all settings, OLEM-Rh consistently outperforms Grad-Rh equipped with a learned graph. This supports the intuition that learning the graph structure introduces edge-level errors that can be harmful to downstream decision-making, and that an explicit graph is not required for strong AUF decision-making performance.

We next examine whether OLEM is a key ingredient in the rehearsal framework by replacing the order-learning component with alternative methods (e.g., CaPS). As shown in Table 3, OLEM-Rh with OLEM achieves higher success probabilities than the same rehearsal procedure coupled with alternative order learners. This indicates that OLEM is particularly well-suited for the rehearsal framework, likely due to its robustness under non-Gaussian settings.

Finally, we compare OLEM-Rh with oracle methods that have access to the true graph, namely Grad-Rh and QWZ23 (Qin et al., 2023). In these comparisons, the true graph is provided to Grad-Rh and QWZ23 but remains unknown to OLEM-Rh. As expected, the oracle method Grad-Rh provides an empirical upper bound. Nevertheless, OLEM-Rh achieves performance comparable to Grad-Rh across all test cases, and outperforms QWZ23 on the four synthetic settings. The superiority of OLEM-Rh to QWZ23 is due to the rigid linear Gaussian assumption of QWZ23, which limits its efficacy in non-Gaussian environments (while ensuring its performance on the linear Gaussian Bermuda dataset).

We further study the effect of sample size on Bermuda. Figure 2a shows absolute performance as a function of sample size, and Figure 2b reports the gap $p_{\text{Grad-Rh}} - p_{\text{OLEM-Rh}}$. Both

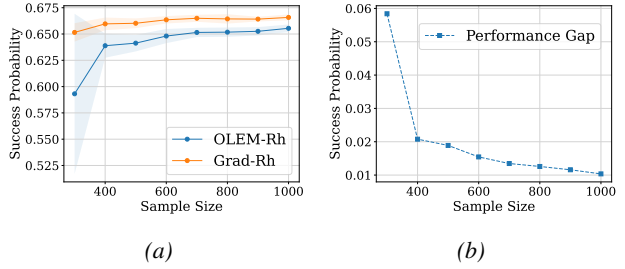


Figure 2. (a) Relation between the success probability and the sample size on the Bermuda dataset. (b) Relation between the performance gap and the sample size on the Bermuda dataset.

methods improve as sample size increases, reflecting more accurate estimation of variable relations. Moreover, the gap narrows with more data: the initial rapid decrease suggests that OLEM-Rh quickly identifies a high-quality order, while the subsequent slower decay indicates that, once the order stabilizes, performance converges as the conditional flow models become the dominant factor in capturing the underlying relations, diminishing the marginal advantage of the graph structure.

5. Conclusion

In this paper, we highlight the unnecessary of the graph structure for decision-making in AUF problems, and demonstrate the sufficiency of the variable order for identifying the influence of decisions from observational data. We propose the first order-based rehearsal learning method OLEM-Rh to learn and exploit the order structure for AUF decision-making. Several modules of this method can be substituted with alternative techniques, providing strong flexibility. Experiments show that OLEM-Rh performs comparably to oracle baselines that have access to the true graph, and outperforms pipelines requiring learning a graph, validating the sufficiency of the order structure for AUF decision-making.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none which we feel must be specifically highlighted here.

References

- Andersson, A. and Bates, N. In situ measurements used for coral and reef-scale calcification structural equation modeling including environmental and chemical measurements, and coral calcification rates in Bermuda from 2010 to 2012 (BEACON project), 2018.
- Anonymity. Non-parametric rehearsal learning via conditional mean embeddings, 2026. Concurrent submission to ICML 2026. Included as Anonymous_concurrent.pdf in supplementary material.
- Ardizzone, L., Bungert, T., Draxler, F., Köthe, U., Kruse, J., Schmier, R., and Sorrenson, P. Framework for Easily Invertible Architectures (FrEIA). 2018–2022.
- Ardizzone, L., Lüth, C., Kruse, J., Rother, C., and Köthe, U. Guided image generation with conditional invertible neural networks. *arXiv preprint arXiv:1907.02392*, 2019.
- Batmaz, Z., Yurekli, A., Bilge, A., and Kaleli, C. A review on deep learning for recommender systems: challenges and remedies. *Artificial Intelligence Review*, 52(1):1–37, 2019.
- Berrett, T. B., Samworth, R. J., and Yuan, M. Efficient multivariate entropy estimation via k -nearest neighbour distances. *The Annals of Statistics*, 47(1):288–318, 2019.
- Boyd, S. and Vandenberghe, L. *Convex optimization*. Cambridge University Press, 2004.
- Bühlmann, P., Peters, J., and Ernest, J. Cam: Causal additive models, high-dimensional order search and penalized regression. *The Annals of Statistics*, 42(6):2526–2556, 2014.
- Chickering, D. M. Optimal structure identification with greedy search. *Journal of Machine Learning Research*, 3: 507–554, 2002.
- Courtney, T. A., Lebrato, M., Bates, N. R., Collins, A., De Putron, S. J., Garley, R., Johnson, R., Molinero, J.-C., Noyes, T. J., Sabine, C. L., and Andersson, A. J. Environmental controls on modern scleractinian coral and reef-scale calcification. *Science Advances*, 3(11): e1701356, 2017.
- Du, W.-B., Qin, T., Wang, T.-Z., and Zhou, Z.-H. Variance-reduced long-term rehearsal learning with quadratic programming reformulation. In *Advances in Neural Information Processing Systems 38*, 2025.
- Erdős, P. and Rényi, A. On the evolution of random graphs. *Publ. Math. Inst. Hung. Acad. Sci.*, 5:17–61, 1960.
- Gardner, J. R., Kusner, M. J., Xu, Z. E., Weinberger, K. Q., and Cunningham, J. P. Bayesian optimization with inequality constraints. In *Proceedings of the 31st International Conference on Machine Learning*, pp. 937–945, 2014.
- Ghoshal, A. and Honorio, J. Learning linear structural equation models in polynomial time and sample complexity. In *Proceedings of the 21st International Conference on Artificial Intelligence and Statistics*, pp. 1466–1475, 2018.
- Glymour, C., Zhang, K., and Spirtes, P. Review of causal discovery methods based on graphical models. *Frontiers in Genetics*, 10:524, 2019.
- Hoyer, P., Janzing, D., Mooij, J. M., Peters, J., and Schölkopf, B. Nonlinear causal discovery with additive noise models. In *Advances in Neural Information Processing Systems 21*, pp. 689–696, 2008.
- Iwasaki, Y. and Simon, H. A. Theories of causal ordering: Reply to de kleer and brown. *Artificial Intelligence*, 29(1):63–72, 1986.
- Kingma, D. and Ba, J. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations*, 2015.
- Kozachenko, L. F. and Leonenko, N. N. Sample estimate of the entropy of a random vector. *Probl. Pered. Inform.*, 23: 95–101, 1987.
- Li, H. Deep learning for natural language processing: advantages and challenges. *National Science Review*, 5(1): 24–26, 2017.
- Pearl, J. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2000.
- Peters, J. and Bühlmann, P. Structural intervention distance for evaluating causal graphs. *Neural Computation*, 27(3): 771–799, 2015.
- Qin, T., Wang, T.-Z., and Zhou, Z.-H. Rehearsal learning for avoiding undesired future. In *Advances in Neural Information Processing Systems 36*, pp. 80517–80542, 2023.

- Qin, T., Wang, T.-Z., and Zhou, Z.-H. Gradient-based non-linear rehearsal learning with multivariate alterations. In *Proceedings of the 39th AAAI Conference on Artificial Intelligence*, pp. 26859–26867, 2025.
- Rolland, P., Cevher, V., Kleindessner, M., Russell, C., Janzing, D., Schölkopf, B., and Locatello, F. Score matching enables causal discovery of nonlinear additive noise models. In *Proceedings of the 39th International Conference on Machine Learning*, pp. 18741–18753, 2022.
- Sachs, K., Perez, O., Pe’er, D., Lauffenburger, D. A., and Nolan, G. P. Causal protein-signaling networks derived from multiparameter single-cell data. *Science*, 308(5721): 523–529, 2005.
- Sanchez, P., Liu, X., O’Neil, A. Q., and Tsaftaris, S. A. Diffusion models for causal discovery via topological ordering. In *Proceedings of the 11th International Conference on Learning Representations*, pp. 20468–20487, 2023.
- Shimizu, S., Hoyer, P. O., Hyvärinen, A., Kerminen, A., and Jordan, M. A linear non-gaussian acyclic model for causal discovery. *Journal of Machine Learning Research*, 7:2003–2030, 2006.
- Simon, H. A. Causal ordering and identifiability. In Hood, W. C. and Koopmans, T. C. (eds.), *Studies in Econometric Methods*, pp. 49–74. John Wiley & Sons, New York, 1953.
- Spirites, P., Glymour, C., and Scheines, R. *Causation, Prediction, and Search*. MIT Press, 2000.
- Sricharan, K., Wei, D., and Hero, A. O. Ensemble estimators for multivariate entropy estimation. *IEEE Transactions on Information Theory*, 59(7):4374–4388, 2013.
- Tao, L., Wang, T.-Z., Jiang, Y., and Zhou, Z.-H. Avoiding undesired future with sequential decisions. In *Proceedings of the 34th International Joint Conference on Artificial Intelligence*, pp. 6245–6253, 2025.
- Teyssier, M. and Koller, D. Ordering-based search: A simple and effective algorithm for learning bayesian networks. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence*, pp. 584–590, 2005.
- Uhler, C., Raskutti, G., Bühlmann, P., and Yu, B. Geometry of the faithfulness assumption in causal inference. *The Annals of Statistics*, pp. 436–463, 2013.
- Voulodimos, A., Doulamis, A., Doulamis, N., and Protopadakis, E. Deep learning for computer vision: A brief review. *Computational Intelligence and Neuroscience*, pp. 1–13, 2018.
- Winkler, C., Worrall, D., Hoogeboom, E., and Welling, M. Learning likelihoods with conditional normalizing flows. *arXiv preprint arXiv:1912.00042*, 2019.
- Xie, F., Li, Z., Wu, P., Zeng, Y., Liu, C., and Geng, Z. Local causal structure learning in the presence of latent variables. *arXiv preprint arXiv:2405.16225*, 2024.
- Xu, Z., Li, Y., Liu, C., and Gui, N. Ordering-based causal discovery for linear and nonlinear relations. In *Advances in Neural Information Processing Systems 37*, pp. 4315–4340, 2024.
- Zanga, A., Ozkirimli, E., and Stella, F. A survey on causal discovery: theory and practice. *International Journal of Approximate Reasoning*, 151:101–129, 2022.
- Zhang, J. On the completeness of orientation rules for causal discovery in the presence of latent confounders. *Artificial Intelligence*, 172(16-17):1873–1896, 2008.
- Zhang, K., Xie, S., Ng, I., and Zheng, Y. Causal representation learning from multiple distributions: A general setting. *arXiv preprint arXiv:2402.05052*, 2024.
- Zheng, X., Aragam, B., Ravikumar, P. K., and Xing, E. P. Dags with no tears: Continuous optimization for structure learning. In *Advances in Neural Information Processing Systems 31*, pp. 9472–9483, 2018.
- Zhou, Z.-H. Rehearsal: learning from prediction to decision. *Frontiers of Computer Science*, 16(4):164352, 2022.

A. Proofs

A.1. Proof of Prop. 3.1

Proposition 3.1. Given a random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$, suppose $\mathbf{V}$ is separated into $\mathbf{X}, \mathbf{Z}, \mathbf{Y}$ with $\mathbf{Z}_A \subseteq \mathbf{Z}$ according to the AUF setting described in Sec. 2. Given an order $\pi \sim \mathcal{G}$, it holds that

$$p(\mathbf{v} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A) = \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) \delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v}) \prod_i p(v_i \mid \mathbf{pred}_i(\pi)).$$

where the product is taken over the V_i -s that are not contained in $\mathbf{X}$ or $\mathbf{Z}_A$. $\mathbf{Pred}_i(\pi) \triangleq \{V_j \mid \pi^{-1}(j) < \pi^{-1}(i)\}$ denotes the predecessors of V_i according to π , and $\mathbf{pred}_i(\pi)$ denotes an assignment to $\mathbf{Pred}_i(\pi)$. $\delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v})$ is a Dirac function satisfying: (1) $\delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) = 0$ if the sub-vector of $\mathbf{v}$ corresponding to $\mathbf{X}$ is not equal to $\mathbf{x}$, and (2) $\int \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) d\mathbf{v} = 1$. $\delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v})$ is similarly defined.

Proof. Recall from Sec. 2 that any variable in $\mathbf{Z}$ or $\mathbf{Y}$ cannot be a parent of a contextual variable X_i . This gives

$$p(\mathbf{v} \mid \mathbf{X} = \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A) = p(\mathbf{v} \mid \mathbf{X} \stackrel{a}{=} \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A).$$

The model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ satisfies the Markov property, i.e. each V_i is independent of its non-descendants given $\mathbf{Pa}_i$ (Pearl, 2000). This gives $p(\mathbf{v}) = \prod_{i \in [d]} p(v_i \mid \mathbf{pa}_i)$. Further, when altering $\mathbf{X}$ and $\mathbf{Z}_A$, for any unaltered V_i , the relation $V_i = f_i(\mathbf{Pa}_i) + \varepsilon_i$ remains unchanged. Therefore it holds that

$$p(\mathbf{v} \mid \mathbf{X} \stackrel{a}{=} \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A) = \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) \delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v}) \prod_i p(v_i \mid \mathbf{pa}_i),$$

where the product is taken over the V_i -s that are not contained in $\mathbf{X}$ or $\mathbf{Z}_A$. Also by the Markov property we have $p(v_i \mid \mathbf{pa}_i) = p(v_i \mid \mathbf{pred}_i(\pi))$, since $\mathbf{Pred}_i(\pi)$ includes all parents of V_i and contains no descendants of V_i . Therefore we obtain

$$p(\mathbf{v} \mid \mathbf{X} \stackrel{a}{=} \mathbf{x}, \mathbf{Z}_A \stackrel{a}{=} \mathbf{z}_A) = \delta_{\mathbf{X}=\mathbf{x}}(\mathbf{v}) \delta_{\mathbf{Z}_A=\mathbf{z}_A}(\mathbf{v}) \prod_i p(v_i \mid \mathbf{pred}_i(\pi)).$$

This finishes the proof. $\square$

A.2. Proof of Prop. 3.2

Proposition 3.2. Under Assumption 3.1, for any $\pi \sim \mathcal{G}$ and $m \in [d]$, it holds that

$$i^* = \arg \max_{i \in \pi[1:m]} h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]}) \text{ implies } i^* \text{ is a sink in } \mathcal{G}^{\pi[1:m]},$$

where $\mathbf{V}_{-i}^{\pi[1:m]} \triangleq \{V_j \mid j \in \pi[1:m], j \neq i\}$.

Proof. For any sink i in $\mathcal{G}^{\pi[1:m]}$, since π is an order of $\mathcal{G}$, it is guaranteed that $\mathbf{V}_{-i}^{\pi[1:m]}$ includes all parents of V_i in $\mathcal{G}$. Also, V_i has no descendants. By the Markov property we have

$$h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]}) = h(V_i \mid \mathbf{Pa}_i) = h(\varepsilon_i).$$

For any non-sink j in $\mathcal{G}^{\pi[1:m]}$, it holds that

$$h(V_j \mid \mathbf{V}_{-j}^{\pi[1:m]}) = h(V_j \mid \mathbf{Des}_j^{\pi[1:m]}, \mathbf{W}_j^{\pi[1:m]}) = h(V_j \mid \mathbf{W}_j^{\pi[1:m]}) - I(V_j; \mathbf{Des}_j^{\pi[1:m]} \mid \mathbf{W}_j^{\pi[1:m]}).$$

Note that $\mathbf{W}_j^{\pi[1:m]}$ includes all parents of V_j in $\mathcal{G}$ and contains no descendants of V_j in $\mathcal{G}$. This gives $h(V_j \mid \mathbf{W}_j^{\pi[1:m]}) = h(V_j \mid \mathbf{Pa}_j) = h(\varepsilon_j)$, and it holds that

$$h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]}) - h(V_j \mid \mathbf{V}_{-j}^{\pi[1:m]}) = h(\varepsilon_i) - h(\varepsilon_j) + I(V_j; \mathbf{Des}_j^{\pi[1:m]} \mid \mathbf{W}_j^{\pi[1:m]}) > 0.$$

Therefore, $i^* = \arg \max_{i \in \pi[1:m]} h(V_i \mid \mathbf{V}_{-i}^{\pi[1:m]})$ implies i^* is a sink node in $\mathcal{G}^{\pi[1:m]}$. $\square$

A.3. Proof of Thm. 3.1

Theorem 3.1. Let $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ be a model satisfying [Assumption 3.1](#), and let $\mathbf{V}$ denote the random vector generated by $\mathcal{M}$. Given the joint distribution p of $\mathbf{V}$ as input, [Algorithm 1](#) returns an order $\pi \sim \mathcal{G}$.

Proof. Denote by π the current order and by Λ the current remaining nodes maintained by [Algorithm 1](#). We prove by induction that for any $t \in [d]$, after the t -th iteration, π is the tail of some order of $\mathcal{G}$ (that is, $\pi(1), \dots, \pi(t)$ are the last t elements of some order of $\mathcal{G}$).

For $t = 1$, before the first iteration, $\Lambda = [d] = \pi[1 : d]$. Plugging $m = d$ into [Prop. 3.2](#) gives that, the node i_1 found in the first iteration is a sink in $\mathcal{G}$. Therefore, after the first iteration, $\pi = (i_1)$ is the tail of some order of $\mathcal{G}$.

Suppose after the t -th iteration ($t \in [d - 1]$), π is the tail of some order $\pi_0 \sim \mathcal{G}$. We then prove this for $t + 1$. Since before the $t + 1$ -th iteration (after the t -th iteration), π is the tail of π_0 , it holds that the current remaining nodes $\Lambda = \pi_0[1 : d - t]$. Plugging $m = d - t$ into [Prop. 3.2](#) gives that, the node i_{t+1} found in the $t + 1$ -th iteration is a sink in $\mathcal{G}^\Lambda$. Therefore, after the $t + 1$ -th iteration, $\pi := (i_{t+1}, \pi)$ is the tail of some order of $\mathcal{G}$.

By induction, for any $t \in [d]$, after the t -th iteration, π is the tail of some order of $\mathcal{G}$. Setting $t = d$ finishes the proof. $\square$

A.4. Proof of Prop. 3.3

Lemma A.1. For any random vectors $\mathbf{X}, \mathbf{Y}, \mathbf{Z}$, it holds that

$$I(\mathbf{Y}; \mathbf{Z} \mid \mathbf{X}) = h(\mathbf{Y} \mid \mathbf{X}) - h(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) + h(\mathbf{X}, \mathbf{Z}).$$

Proof. $I(\mathbf{Y}; \mathbf{Z} \mid \mathbf{X}) = h(\mathbf{Y} \mid \mathbf{X}) - h(\mathbf{Y} \mid \mathbf{X}, \mathbf{Z}) = h(\mathbf{Y} \mid \mathbf{X}) - h(\mathbf{X}, \mathbf{Y}, \mathbf{Z}) + h(\mathbf{X}, \mathbf{Z})$. $\square$

Proposition 3.3. Under the linear Gaussian setting, [Assumption 3.1](#) reduces to [Assumption 3.2](#), and [Assumption 3.3](#) implies [Assumption 3.1](#).

Proof. Consider any random vector $\mathbf{V}$ generated by the model $\mathcal{M} = (\mathcal{G}, \{f_i\}_i, \{\varepsilon_i\}_i)$ under the linear Gaussian setting:

$$f_i(\mathbf{P}\mathbf{a}_i) = \sum_{V_j \in \mathbf{P}\mathbf{a}_i} w_{ji} V_j, \quad \varepsilon_i \sim \mathcal{N}(0, \sigma_i^2).$$

For any $\pi \sim \mathcal{G}$, $m \in [d]$, and for any $i, j \in \pi[1 : m]$ such that i is a sink in $\mathcal{G}^{\pi[1:m]}$, while j is not, we show that the inequality in [Assumption 3.1](#) reduces to that in [Assumption 3.2](#).

Denote by Σ the covariance matrix of $\mathbf{V}^{\pi[1:m]}$, by $|\Sigma|$ the determinant of Σ , and by M_{jj} the determinant of the sub-matrix obtained by removing the row and column corresponding to V_j in Σ . By [Lemma A.1](#), it holds that

$$I(V_j; \mathbf{Des}_j^{\pi[1:m]} \mid \mathbf{W}_j^{\pi[1:m]}) = h(V_j \mid \mathbf{W}_j^{\pi[1:m]}) - h(\mathbf{W}_j^{\pi[1:m]}, V_j, \mathbf{Des}_j^{\pi[1:m]}) + h(\mathbf{W}_j^{\pi[1:m]}, \mathbf{Des}_j^{\pi[1:m]}).$$

From the proof of [Prop. 3.2](#) we have

$$h(V_j \mid \mathbf{W}_j^{\pi[1:m]}) = h(\varepsilon_j) = (1/2) \ln(2\pi e \sigma_j^2).$$

Also, noting that $\mathbf{V}^{\pi[1:m]} = \mathbf{W}_j^{\pi[1:m]} \cup \{V_j\} \cup \mathbf{Des}_j^{\pi[1:m]}$, we have

$$\begin{aligned} h(\mathbf{W}_j^{\pi[1:m]}, V_j, \mathbf{Des}_j^{\pi[1:m]}) &= (1/2) \ln((2\pi e)^m |\Sigma|), \\ h(\mathbf{W}_j^{\pi[1:m]}, \mathbf{Des}_j^{\pi[1:m]}) &= (1/2) \ln((2\pi e)^{m-1} M_{jj}). \end{aligned}$$

Therefore it holds that

$$I(V_j; \mathbf{Des}_j^{\pi[1:m]} \mid \mathbf{W}_j^{\pi[1:m]}) = \frac{1}{2} \ln \frac{\sigma_j^2 M_{jj}}{|\Sigma|}.$$

Denote by Ω the inverse matrix of Σ , and by Σ^* the adjugate matrix of Σ . Let Ω_{jj} denote the diagonal element of Ω corresponding to V_j , and define Σ_{jj}^* similarly. Note that $\Omega = \Sigma^* / |\Sigma|$. This gives $\Omega_{jj} = \Sigma_{jj}^* / |\Sigma| = M_{jj} / |\Sigma|$. Plugging this into the equation above gives

$$I(V_j; \text{Des}_j^{\pi[1:m]} | \mathbf{W}_j^{\pi[1:m]}) = \frac{1}{2} \ln(\sigma_j^2 \Omega_{jj}).$$

By Prop.2 of Ghoshal & Honorio (2018), $\Omega_{jj} = \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2$. This gives

$$\begin{aligned} I(V_j; \text{Des}_j^{\pi[1:m]} | \mathbf{W}_j^{\pi[1:m]}) > h(\varepsilon_j) - h(\varepsilon_i) \quad &\text{iff} \quad \frac{1}{2} \ln(\sigma_j^2 \sigma_i^{-2}) < \frac{1}{2} \ln \sigma_j^2 \left(\sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2 \right) \\ &\text{iff} \quad \sigma_i^{-2} < \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2. \end{aligned}$$

Therefore, Assumption 3.1 reduces to Assumption 3.2 under the linear Gaussian setting.

Then we prove Assumption 3.3 implies Assumption 3.2. For any $\pi \sim \mathcal{G}$, $m \in [d]$, suppose (1) of Assumption 3.3 holds, then for any $i, j \in \pi[1 : m]$ such that i is a sink in $\mathcal{G}^{\pi[1:m]}$, while j is not, we have $\sigma_i > \sigma_j$, and this gives

$$\sigma_i^{-2} < \sigma_j^{-2} \leq \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2.$$

Also, suppose (2) of Assumption 3.3 holds, then for any $i, j \in \pi[1 : m]$ such that i is a sink in $\mathcal{G}^{\pi[1:m]}$, while j is not, noting that $\frac{\partial f_i(\mathbf{pa}_i)}{\partial v_j} = w_{ji}$, it holds that

$$\sigma_i^{-2} \leq \sigma_{\min}^{-2} < \sigma_j^{-2} + \sum_{l: V_j \in \mathbf{Pa}_l^{\pi[1:m]}} \sigma_l^{-2} w_{jl}^2.$$

Therefore, Assumption 3.3 implies Assumption 3.2 under the linear Gaussian setting, and it follows that Assumption 3.3 implies Assumption 3.1 under the linear Gaussian setting. $\square$

B. Experiments

B.1. Implementation details

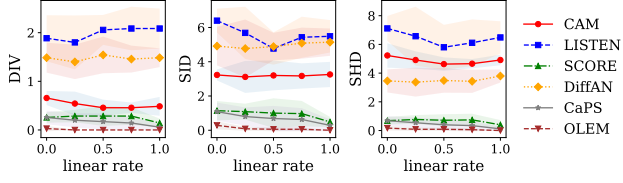
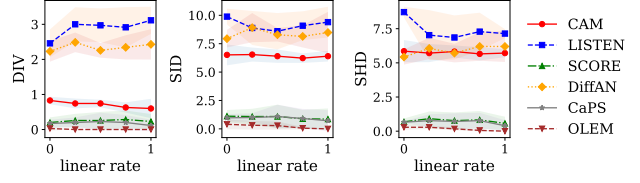
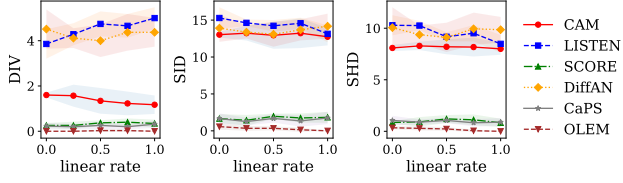
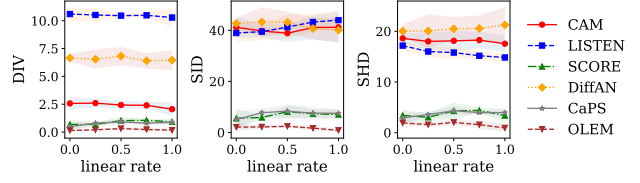
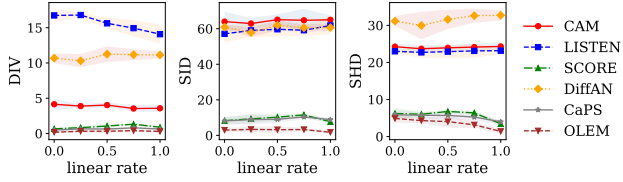
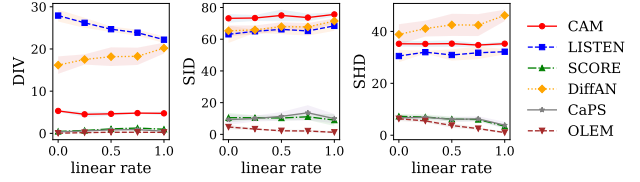
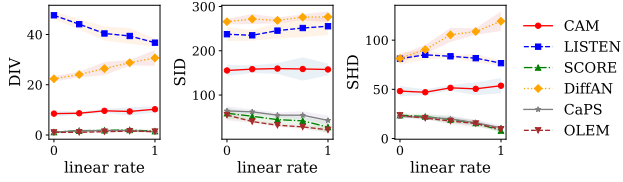
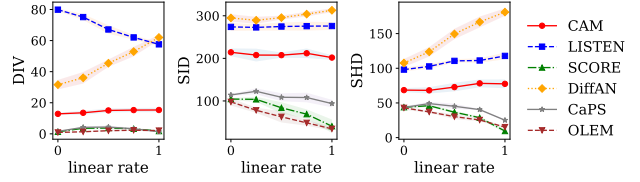
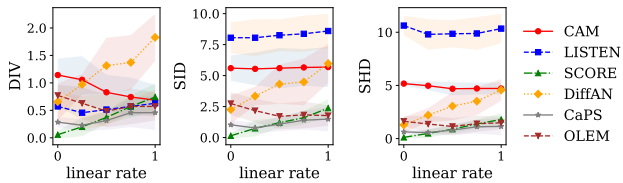
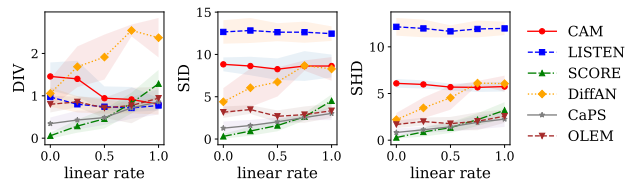
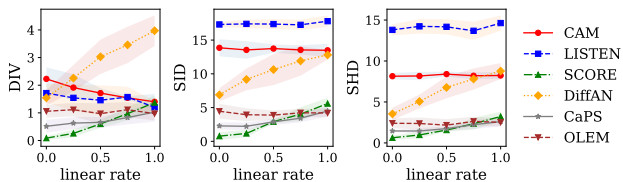
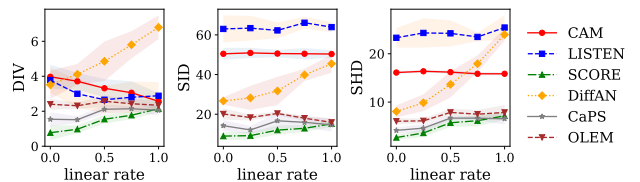
Order Learning. As presented in Algorithm 1, OLEM recursively learns the order from the joint distribution, where the joint distribution is only used to compute the conditional entropy terms. Since we only have access to observational data in practice, we estimate the required conditional entropy terms using the Kozachenko–Leonenko estimator (Kozachenko & Leonenko, 1987). Notably, OLEM is parameter-free and requires no hyperparameter tuning.

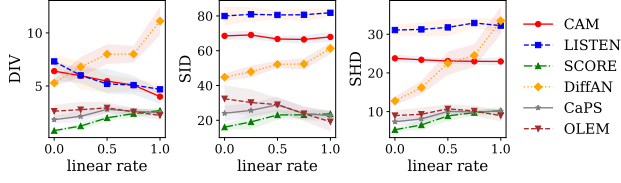
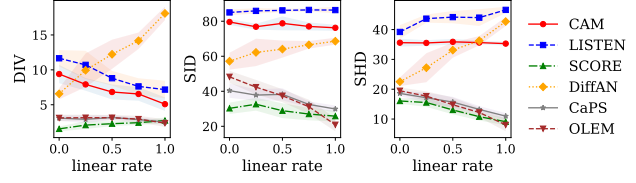
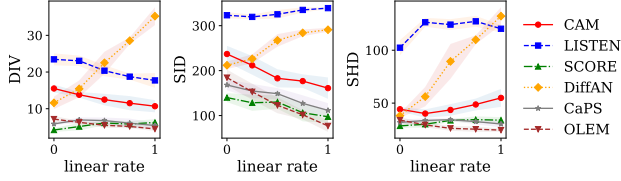
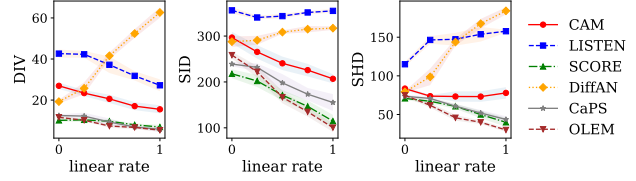
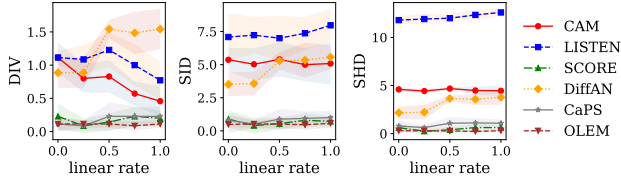
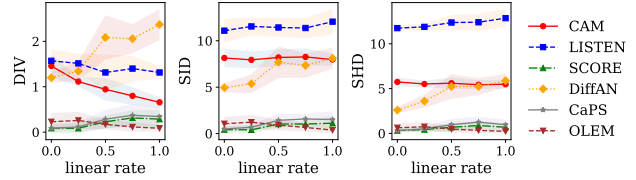
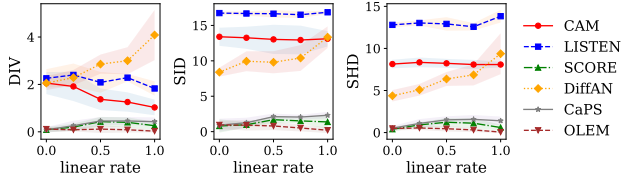
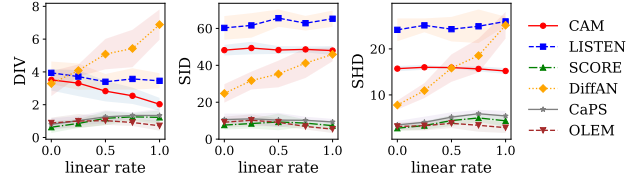
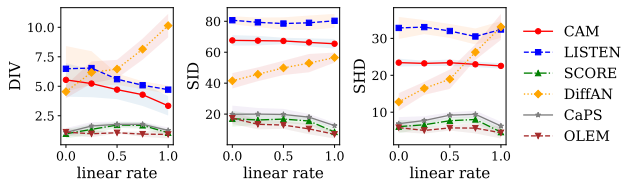
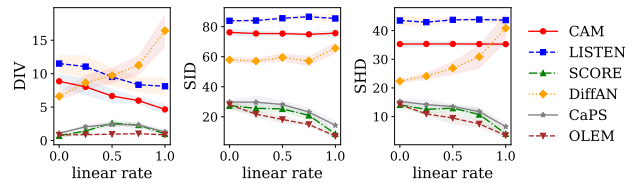
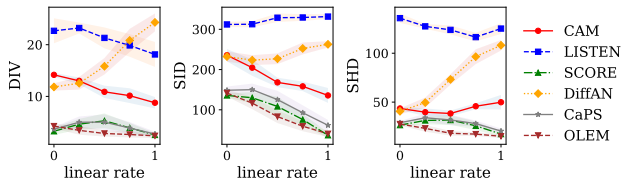
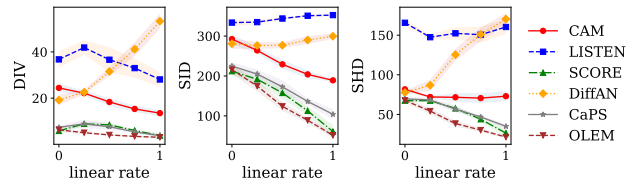
AUF Decision-making. OLEM-Rh leverages the conditional flow model (Winkler et al., 2019) to construct the post-decision distribution sampler, where the flow model is implemented using the conditional invertible neural network (cINN) (Ardizzone et al., 2019). Specifically, the cINN in our experiment consists of 16 sequentially chained blocks, and is trained using the Adam optimizer (Kingma & Ba, 2015) with a learning rate of 0.001 and a batch size of 256. During the training process, we employ a 70/30 train-validation split and use the validation set for early stopping. Detailed architectural and training specifications of the cINN follow Ardizzone et al. (2018–2022; 2019).

With the constructed sampler, OLEM-Rh solves the problem (3) using Adam, whose learning rate is set to 0.5 in our experiments. The size n of the samples drawn from the constructed sampler is a hyperparameter, and is fixed at $n = 1000$ across all experiments to ensure consistency.

B.2. Additional results

We present additional results of the synthetic experiments in Sec. 4.1. The noise variables follow beta, Gaussian, or exponential distributions. We vary the dimensionality $d \in \{5, 10, 20\}$ and the edge density $p \in \{0.3, 0.5, 0.8\}$. Although the results for $d = 10$, $p = 0.3$ with beta or Gaussian noise are shown in Figure 1, we reproduce them here to facilitate direct comparison. The setting $d = 20$, $p = 0.8$ is omitted due to the excessive number of edges.


 Figure 3. $d = 5$, $p = 0.3$ with beta noise

 Figure 4. $d = 5$, $p = 0.5$ with beta noise

 Figure 5. $d = 5$, $p = 0.8$ with beta noise

 Figure 6. $d = 10$, $p = 0.3$ with beta noise

 Figure 7. $d = 10$, $p = 0.5$ with beta noise

 Figure 8. $d = 10$, $p = 0.8$ with beta noise

 Figure 9. $d = 20$, $p = 0.3$ with beta noise

 Figure 10. $d = 20$, $p = 0.5$ with beta noise

 Figure 11. $d = 5$, $p = 0.3$ with Gaussian noise

 Figure 12. $d = 5$, $p = 0.5$ with Gaussian noise

 Figure 13. $d = 5$, $p = 0.8$ with Gaussian noise

 Figure 14. $d = 10$, $p = 0.3$ with Gaussian noise


 Figure 15. $d = 10$, $p = 0.5$ with Gaussian noise

 Figure 16. $d = 10$, $p = 0.8$ with Gaussian noise

 Figure 17. $d = 20$, $p = 0.3$ with Gaussian noise

 Figure 18. $d = 20$, $p = 0.5$ with Gaussian noise

 Figure 19. $d = 5$, $p = 0.3$ with exponential noise

 Figure 20. $d = 5$, $p = 0.5$ with exponential noise

 Figure 21. $d = 5$, $p = 0.8$ with exponential noise

 Figure 22. $d = 10$, $p = 0.3$ with exponential noise

 Figure 23. $d = 10$, $p = 0.5$ with exponential noise

 Figure 24. $d = 10$, $p = 0.8$ with exponential noise

 Figure 25. $d = 20$, $p = 0.3$ with exponential noise

 Figure 26. $d = 20$, $p = 0.5$ with exponential noise